

## Week 2 · Student Notes

AI531C Digital Image Analysis · Tue 11 – Wed 12 Aug 2026 · Unit I · Lectures 4, 5, 6

*Everything taught in this week, in one document. The per-lecture handouts are identical in content — use whichever you prefer.*

### What is in this bundle

- Lecture 4 — Pixel Neighbourhoods, Adjacency, Connectivity and Distance Measures
- Lecture 5 — Colour Spaces: RGB, HSV, YCbCr and CIE-Lab
- Lecture 6 — Point Operations and Intensity Transformations

### This week at a glance

Sessions — Tue 11 – Wed 12 Aug 2026.

Tutorial — T2 (Tue 11 Aug).

Pre-class preparation — PCM-2.

Released after the last class of the week: Wed 12 Aug 2026, 18:00.

*How to use this bundle — read the lecture section on the evening of the class while the argument is fresh, attempt the check-your-understanding questions before the tutorial, and leave the practice questions until you can do them unaided. No answer keys are printed: the reasoning is worked through in the tutorial.*

# Lecture 4 · Pixel Neighbourhoods, Adjacency, Connectivity and Distance Measures

AI531C Digital Image Analysis · Unit I · CO1 · M.Tech. Artificial Intelligence, Department of CSE, DTU

*When are two pixels 'together'? The formal machinery behind every object count, region and measurement in this course*

## What you should already know

This lecture continues from Lecture 3, “Sampling, Quantisation, Aliasing and Quantisation Artefacts”. Revise that material first — today's arguments assume it.

Reading in the course notes: Comprehensive Notes L5; Vol. I ch. 5.

## Learning outcomes

After studying this lecture you should be able to:

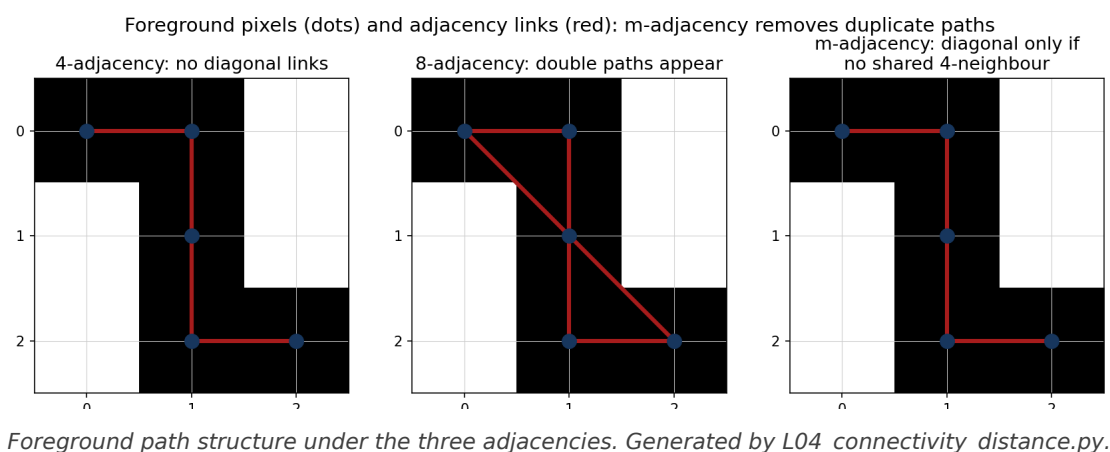
## Core concepts

### Neighbourhoods: N4, ND, N8

For a pixel  $p$  at coordinates  $(x, y)$  —  $N4(p) = \{(x+1, y), (x-1, y), (x, y+1), (x, y-1)\}$  — the 4 edge-sharing neighbours;  $ND(p) = \{(x+1, y+1), (x+1, y-1), (x-1, y+1), (x-1, y-1)\}$  — the 4 diagonal neighbours;  $N8(p) = N4(p) \cup ND(p)$  — all 8 surrounding pixels.

Boundary convention — Pixels outside the image are not neighbours — a border pixel has a truncated neighbourhood. Every library you use (OpenCV, scikit-image, PyTorch padding) makes an explicit choice here; you met the consequences as border handling in L3 and will again in convolution (L8).

## Figure 1: Adjacency: 4, 8, and the m-adjacency repair



## How to read this figure

What to notice — Under 8-adjacency the same two pixels are linked by multiple redundant paths (middle panel).

Why it happens — A diagonal link is allowed even when the two pixels already share a 4-neighbour.

What it means — m-adjacency permits a diagonal link only if the pixels share no foreground 4-neighbour — paths become unique.

Why it matters — Boundary tracing and thinning need unique paths (L17).

## Distance measures on the pixel grid

Problem: quantify how far apart two pixels  $p = (x, y)$  and  $q = (s, t)$  are. The answer depends on which moves we count as one step. Assume integer grid coordinates.

$$D_E = \sqrt{(x-s)^2 + (y-t)^2} \quad D_4 = |x-s| + |y-t| \quad D_8 = \max(|x-s|, |y-t|)$$

## What each symbol means

- $p = (x, y)$ ,  $q = (s, t)$  — pixel coordinates (row, column)  $D_E$  — Euclidean: straight-line distance
- $D_4$  — city-block: steps via 4-neighbours only  $D_8$  — chessboard: steps via 8-neighbours (king moves)

## How to read this

What it means —  $D_4$  and  $D_8$  count minimal path lengths under 4- and 8-adjacency respectively. Ordering always holds:  $D_8 \leq D_E \leq D_4$  — the more moves allowed, the shorter the path.

## Worked example

Given —  $p = (1, 1)$  and  $q = (4, 5)$  on a pixel grid.

Find —  $D_E$ ,  $D_4$  and  $D_8$  between  $p$  and  $q$ ; verify the ordering  $D_8 \leq D_E \leq D_4$ .

Formula —  $D_E = \sqrt{(x-s)^2 + (y-t)^2}$ ;  $D_4 = |x-s| + |y-t|$ ;  $D_8 = \max(|x-s|, |y-t|)$

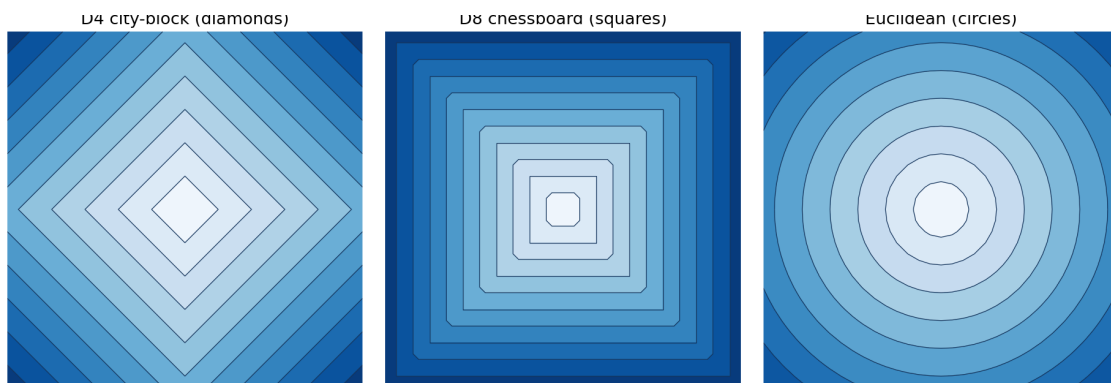
## Solution

Substitution —  $|x-s| = |1-4| = 3$ ;  $|y-t| = |1-5| = 4$

Calculation —  $D_E = \sqrt{(9+16)} = \sqrt{25}$ ;  $D_4 = 3+4$ ;  $D_8 = \max(3,4)$  ANSWER:  $D_E = 5.0$ ,  $D_4 = 7$ ,  $D_8 = 4$

What it means —  $4 \leq 5 \leq 7$  ✓. A  $D_8$  path uses 4 king-moves; the  $D_4$  path needs all 7 axis steps. Verified programmatically in `L04_connectivity_distance.py`.

## Figure 2: What equidistance looks like



*Iso-distance contours from a centre pixel:  $D_4$ ,  $D_8$ ,  $D_E$ . Generated by `L04_connectivity_distance.py`.*

## How to read this figure

What to notice — Equal-distance sets are diamonds ( $D_4$ ), squares ( $D_8$ ) and circles ( $D_E$ ).

Why it happens — Each metric's unit ball is the set reachable in one 'step' — the geometry of the allowed moves.

What it means — A 'disk' of radius  $r$  means different pixel sets under different metrics.

Why it matters — Distance transforms, dilation radii (L17) and watershed markers (L16) change shape with the metric — declare it in your code.

## The code behind the results

```
import cv2, numpy as np
B = (img > 128).astype(np.uint8)
n4, lab4 = cv2.connectedComponents( B, connectivity=4)
n8, lab8 = cv2.connectedComponents( B, connectivity=8)
print(n4 - 1, n8 - 1)
# background excluded
```

Parameters used —  $\text{connectivity} \in \{4, 8\}$ . Full program: `L04_connectivity_distance.py`

What it means — 7 components vs 4 on the same image — the parameter IS the definition of 'object'.

Something to think about — Thin diagonal structures: 4-connectivity shatters them; 8 keeps them whole.

## Choosing adjacency and distance for a task

| Task                                               | Sensible choice             | Why                                                                |
|----------------------------------------------------|-----------------------------|--------------------------------------------------------------------|
| Counting compact objects (cells, coins)            | 8-connectivity              | Small diagonal touches should not split an object                  |
| Tracing thin diagonal structures (cracks, vessels) | 8- or m-connectivity        | 4-adjacency shatters diagonal runs                                 |
| Boundary following / thinning                      | m-adjacency                 | Unique paths guarantee clean termination                           |
| Foreground vs background pair                      | FG 8 + BG 4 (or vice versa) | Same adjacency for both creates the crossing paradox               |
| Distance transform for watershed markers           | D_E (DIST_L2)               | Isotropic basins; D <sub>4</sub> /D <sub>8</sub> bias marker shape |

*Key takeaway: adjacency and distance are modelling decisions — record them alongside thresholds in every experiment.*

## Core concepts (continued): Failure case: the connectivity paradox

Setup — A one-pixel-wide diagonal ring of foreground pixels encloses a background pixel.

The paradox — With 8-adjacency for both foreground and background: the ring is a closed connected curve, yet the inside background still connects to the outside background diagonally — a closed curve that fails to enclose anything.

The repair — Use complementary adjacencies: foreground 8 ↔ background 4 (or foreground 4 ↔ background 8). Every correct hole-filling and contour algorithm does this internally (L17).

The lesson — Definitions are not pedantry — an unstated adjacency is a latent bug that surfaces as impossible topology in your masks.

## Where this is used — your project

Segmentation lane (and every mask-producing lane) — component counting = your object count metric — report the connectivity used; mask cleanup: remove components below an area threshold (WithStats); evaluation: a predicted mask split into fragments changes counts under 4- vs 8-connectivity — fix the convention before computing Dice/IoU per object (L37); change detection: connected regions of the difference map are your 'changes' — their number is adjacency-dependent. Add one line to your dataset card: "connectivity: 8 (objects), 4 (background)".

Quick wins — Distance transform → markers for watershed (T7); per-component area histogram = instant size QC for your data.

## Summary

- 'Object' is defined by adjacency, not by the image — declare it. m-adjacency exists to make paths unique; FG/BG need complementary adjacencies. D<sub>4</sub>, D<sub>8</sub>, D\_E are different unit balls: diamond, square, circle — pick to match the task.

## Key formulae

$$N8(p) = N4(p) \cup ND(p) \quad D_E = \sqrt{(x-s)^2 + (y-t)^2} \quad D_4 = |x-s| + |y-t| \quad D_8 = \max(|x-s|, |y-t|)$$

$$\text{Ordering: } D_8 \leq D_E \leq D_4$$

## What comes next

Tutorial T2 — today 2:00–3:00 PM (studio) — Sampling, quantisation, colour and contrast studio (PCM-2 artefacts in hand). Add-on from today: run connectedComponents at both connectivities on

your own thresholded image and explain any count difference in one sentence.

Before tomorrow (L5-L6 double lecture) — From PCM-2: bring your HSV/Lab channel observations — L5 builds directly on them.

Logistics — L5 colour spaces + L6 intensity transforms, Wed 10 AM–12 PM. PCM-3 (kernel prediction card) releases Friday 5 PM.

## Check your understanding

Work these before the tutorial. They are the same questions used in class; the reasoning is discussed there, so attempt them first.

### 1. Micro-challenge: count before you compute

For  $V = \{1\}$ , count the connected components: 0 1 0 0 1 0 1 0 1 0 0 0 0 1 0 1 0 1 0 0 1 1 0 0 0 (a) under 4-adjacency; (b) under 8-adjacency. Then justify one count to your neighbour using the definitions.

## Practice questions

Attempt these unaided, then compare with the discussion in your tutorial session.

Q1 (4 marks, remember/understand): Define  $N_4$ ,  $N_D$ ,  $N_8$  and state the  $m$ -adjacency rule. Why does  $m$ -adjacency exist?

Q2 (6 marks, apply): For  $p = (2, 3)$  and  $q = (7, 9)$ , compute  $D_E$ ,  $D_4$  and  $D_8$ , and give one pixel pair for which all three distances are equal.

Q3 (10 marks, analyse): A crack-detection system reports 40 cracks where inspectors count 12. The mask is one pixel wide and largely diagonal. Diagnose the discrepancy, state the repair, and explain what happens to background connectivity after your repair.

## Further reading

GW4 Ch.2 Sec.2.5; GW4 Ch.9 connectivity prerequisites

Course notes: Comprehensive Notes L5; Vol. I ch. 5.

Runnable code for every figure and number in this lecture: 06\_CODE\_AND\_DEMOS (see the file names quoted beside each figure).

*AI531C Digital Image Analysis · student handout · every figure and numerical result in this document is the output of course code you can run yourself.*

# Lecture 5 · Colour Spaces: RGB, HSV, YCbCr and CIE-Lab

AI531C Digital Image Analysis · Unit I · CO1 · M.Tech. Artificial Intelligence, Department of CSE, DTU

*Colour is information — and a trap. Choosing the space in which to compute is the first real design decision of most pipelines*

## What you should already know

This lecture continues from Lecture 4, “Pixel Neighbourhoods, Adjacency, Connectivity and Distance Measures”. Revise that material first — today's arguments assume it.

Reading in the course notes: Comprehensive Notes L6; Vol. I ch. 6.

## Learning outcomes

After studying this lecture you should be able to:

## Luma: the engineered brightness axis

Problem: reduce an RGB triple to a single brightness value that matches human perception. Assume display-referred 8-bit sRGB primaries (BT.601 weights).

$$Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

## What each symbol means

- R, G, B — channel intensities, 0–255 Y — luma (perceived brightness), 0–255
- Weights — human sensitivity: green  $\approx$  59%, red  $\approx$  30%, blue  $\approx$  11%
- Cb, Cr — the two chroma differences completing YCbCr

## How to read this

What it means — The eye is most sensitive to green — equal-weight averaging  $(R+G+B)/3$  gets brightness visibly wrong. Separating Y from Cb/Cr lets JPEG and video subsample chroma 2–4× with little visible loss.

## Core concepts

### Four spaces, four intentions

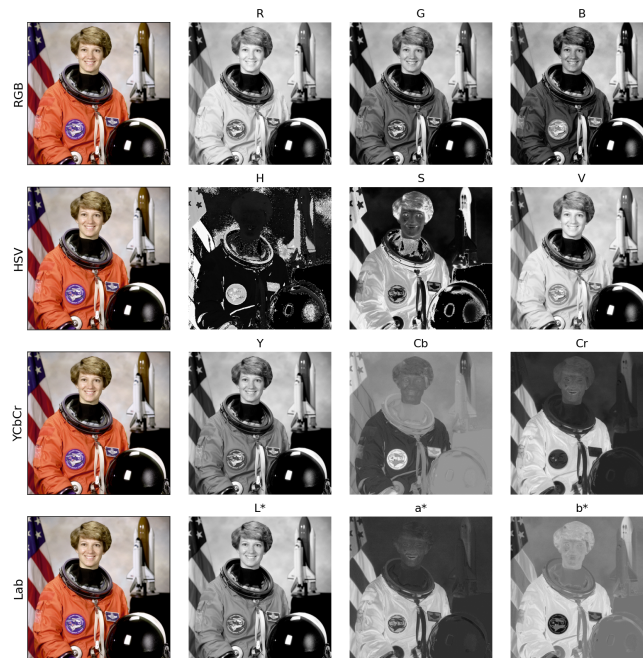
RGB — the device space — What sensors record and displays emit. Best for I/O; poor for analysis because all three axes mix brightness with colour.

HSV — the intuition space — Hue (what colour), Saturation (how pure), Value (how bright) — cylindrical re-arrangement of RGB. Best for colour-based rules and thresholds.

YCbCr — the compression space — Luma + two chroma differences. Best when brightness and colour must be processed or transmitted separately (JPEG, video, skin detection).

CIE-Lab — the perceptual space — L\* lightness, a\* green–red, b\* blue–yellow, designed so equal Euclidean steps  $\approx$  equal perceived differences. Best for measuring colour difference.

## Figure 1: Reading the channel decompositions



*astronaut() decomposed in four spaces. Generated by L05\_colour\_spaces.py.*

### How to read this figure

What to notice — V and Y look like the grayscale photo; H is flat within objects; Cb/Cr carry little fine detail.

Why it happens — Most scene detail is brightness detail; colour varies slowly across surfaces.

What it means — Structure lives in the luma axis; identity of colour lives in chroma/hue.

Why it matters — Edges and filters run on Y or V; colour rules run on H or Cb/Cr.

### Worked example

Given — (a) Pixel (R, G, B) = (200, 120, 80). (b) Two Lab colours: (52, 10, 20) and (50, 14, 17).

Find — (a) Luma Y. (b) Colour difference  $\Delta E$  and whether it is visible.

Formula —  $Y = 0.299R + 0.587G + 0.114B$ ;  $\Delta E = \sqrt{(\Delta L^2 + \Delta a^2 + \Delta b^2)}$

### Solution

Substitution — (a)  $Y = 0.299 \cdot 200 + 0.587 \cdot 120 + 0.114 \cdot 80$ . (b)  $\Delta L = 2$ ,  $\Delta a = -4$ ,  $\Delta b = 3$

Calculation — (a)  $Y = 59.8 + 70.44 + 9.12$ . (b)  $\Delta E = \sqrt{(4 + 16 + 9)} = \sqrt{29}$  ANSWER:  $Y \approx 139$ ;  $\Delta E \approx 5.39$  — clearly visible (JND  $\approx 2.3$ )

What it means — A  $\Delta E$  of  $\sim 5$  fails print/display QC. The same two colours compared by RGB Euclidean distance would give a number with no perceptual meaning — Lab is the space where distance means difference.

### The code behind the results

```
hsv = cv2.cvtColor(rgb, cv2.COLOR_RGB2HSV) H, S = hsv[...,0], hsv[...,1] red = ((H < 12) | (H > 168)) & (S > 60)
dark = (rgb * 0.35).astype(np.uint8)
# dim it # ...same mask on dark image still fires
```

Parameters used — hue window 12/168 (red wraps around 0);  $S > 60$  floor. Full program: L05\_colour\_spaces.py

What it means — The rule is written once, in the right space, and survives a 65% light drop.

Something to think about — Remove the  $S > 60$  floor? Grey pixels acquire random hue — the next slide's failure.

## Choosing the space: summary table

| Space   | Axes separate...                          | Best for                                    | Watch out for                             |
|---------|-------------------------------------------|---------------------------------------------|-------------------------------------------|
| RGB     | Nothing — brightness in all axes          | Capture, display, per-channel ops           | Rules break under illumination change     |
| HSV     | Colour angle from brightness              | Colour thresholds, quick rules              | Hue undefined at low S/V; H stored 0–179  |
| YCbCr   | Luma from chroma                          | Compression, skin detection, filtering on Y | Chroma is coarse — not for fine colour QC |
| CIE-Lab | Perceptual lightness from opponent colour | Colour difference ( $\Delta E$ ), matching  | Assumes a reference illuminant (D65)      |

*Key takeaway: convert deliberately, compute in the space whose axes match your question, convert back for display.*

## Where this is used — your project

Every lane makes at least one colour-space decision — low-light enhancement: enhance V or  $L^*$ , keep H — enhancing RGB channels independently shifts colours (preview of L33's colour-consistency criterion); denoising/deblurring: filter Y strongly, chroma gently — matches noise statistics and vision; segmentation: try hue/chroma features before training anything; change detection: difference in Lab ( $\Delta E$  per pixel) beats RGB differencing under lighting drift. Add to your dataset card: working colour space + conversion call used.

Five-minute experiment — Run your PCM-2 image through the channel decomposition; note which single channel best isolates your object of interest.

## Summary

- Colour spaces are engineered separations: device, intuition, compression, perception. Structure lives in luma; colour identity lives in hue/chroma — compute where your question lives. Hue is only meaningful when chroma is — guard every hue rule with a saturation floor.

## Key formulae

$$Y = 0.299R + 0.587G + 0.114B \text{ (BT.601)} \quad V = M; \quad S = (M - m)/M; \quad H \text{ from 6-sector rule } \Delta E = \sqrt{(\Delta L^2 + \Delta a^2 + \Delta b^2)}; \quad JND \approx 2.3$$

## What comes next

Immediately next (L6, same room) — Point operations and intensity transformations — applied on the V/Y channels you met today.

This week — PCM-3 (kernel prediction card, ~30 min) releases Friday 5 PM, due Monday 8 PM — it feeds L7, L8 and Tuesday's filtering studio T3.

Logistics — Tutorial T3: Tue 18 Aug, 2–3 PM — spatial filtering and convolution studio. Bring: your PCM-2 channel observations to T3 as reference material.

## Check your understanding

Work these before the tutorial. They are the same questions used in class; the reasoning is discussed there, so attempt them first.

### 1. Which colour space?

Choose a working space (and the axis you'd threshold) for each task: A. Detect ripe (red) tomatoes on a conveyor under lights that flicker in brightness. B. Compress drone footage for transmission with least visible loss. C. Verify that two production batches of paint match. D. Run Canny edge detection on a colour photo.

## 2. Predict: the grey T-shirt

A pixel is nearly grey:  $(R, G, B) = (128, 126, 130)$ , plus sensor noise of  $\pm 3$  per channel. Predict: 1. Roughly what are  $S$  and  $C = M - m$  for this pixel? 2. What happens to its computed hue from frame to frame as noise flips which channel is max? 3. What does this do to a hue-threshold mask on grey/desaturated objects — and what guard fixes it?

## Practice questions

Attempt these unaided, then compare with the discussion in your tutorial session.

Q1 (4 marks, remember/understand): Name the four course colour spaces and state, for each, the one task it serves best and why.

Q2 (6 marks, apply): Compute luma  $Y$  for  $(R, G, B) = (50, 100, 150)$ , and  $\Delta E$  between Lab colours  $(60, -5, 10)$  and  $(63, -1, 12)$ . State whether the colour difference is visible.

Q3 (10 marks, analyse): A fruit-sorting system thresholds hue and works in the morning but misfires in the afternoon on shaded, dusty (desaturated) fruit. Diagnose both failure mechanisms and redesign the rule.

## Further reading

GW4 Ch.6; Szeliski Ch.2 (photometric) & Ch.3 (point operators); Elgendy Ch.1.4

Course notes: Comprehensive Notes L6; Vol. I ch. 6.

Runnable code for every figure and number in this lecture: 06\_CODE\_AND\_DEMOS (see the file names quoted beside each figure).

*AI531C Digital Image Analysis · student handout · every figure and numerical result in this document is the output of course code you can run yourself.*

# Lecture 6 · Point Operations and Intensity Transformations

AI531C Digital Image Analysis · Unit I · CO1 · M.Tech. Artificial Intelligence, Department of CSE, DTU

*One curve, applied to every pixel: negative, logarithmic, gamma and contrast stretching — the smallest tools in the box, used everywhere*

## What you should already know

This lecture continues from Lecture 5, “Colour Spaces: RGB, HSV, YCbCr and CIE-Lab”. Revise that material first — today's arguments assume it.

Reading in the course notes: Comprehensive Notes L7; Vol. I ch. 7.

## Learning outcomes

After studying this lecture you should be able to:

## The point-operation family

Problem: remap stored intensities  $r$  to displayed/processed intensities  $s$  with a single function  $T$ . Assume grayscale (or the V/Y channel),  $r$  normalised to  $[0, 1]$  unless stated.

$$s = (L-1) - r \quad s = c \cdot \log(1 + r) \quad s = c \cdot r^\gamma$$

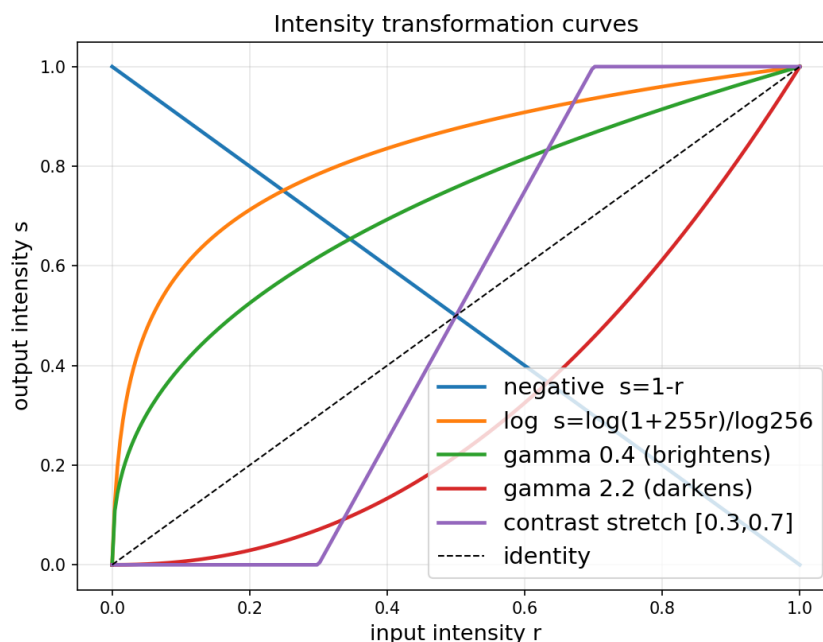
## What each symbol means

- $r, s$  — input / output intensity  $L$  — number of levels (256 for 8-bit)  $c$  — scaling constant (chosen to fill the output range)  $\gamma$  — gamma exponent:  $\gamma < 1$  brightens,  $\gamma > 1$  darkens

## How to read this

What it means — All three are monotone curves — they reorder no intensities, only re-space them. Log compresses a huge dynamic range (Fourier spectra in L9!); gamma matches display and perception; negative flips film and mammograms.

## Figure 1: Reading the curve family



The transform curves on one axis. Generated by `L06_intensity_transforms.py`.

## How to read this figure

What to notice — Gamma 0.4 rises steeply at the dark end; gamma 2.2 is flat there; the stretch is a straight ramp between two thresholds.

Why it happens — Slope of T at input  $r$  = local contrast gain at that intensity.

What it means — Steep where you want detail revealed; flat where you sacrifice it.

Why it matters — Choose the curve by asking: which intensities deserve the slope?

## Worked example

Given — 8-bit pixel  $r = 30$  in an under-exposed image; gamma correction with  $\gamma = 0.4$ , full-range scaling.

Find — Output value  $s$ ; and the output for the same pixel under the full-range log transform.

Formula —  $s = 255 \cdot (r/255)^\gamma$ ;  $s_{\log} = 45.99 \cdot \ln(1 + r)$

### Solution

Substitution —  $r/255 = 0.1176$ ;  $s = 255 \cdot 0.1176^{0.4}$ ;  $s_{\log} = 45.99 \cdot \ln(31)$

Calculation —  $0.1176^{0.4} = 0.4248 \rightarrow s = 108.3$ ;  $s_{\log} = 45.99 \times 3.434 = 157.9$  ANSWER:  $s_{\gamma} \approx 108$ ;  $s_{\log} \approx 158$

What it means — Both lift the dark pixel; log lifts harder. The price is paid at the bright end, where log's flat tail crushes highlight contrast — check the curve before choosing.

## The code behind the results

```
r = np.arange(256, dtype=np.float64)
neg = (255 - r).astype(np.uint8)
logT = (45.99*np.log1p(r)).astype(np.uint8)
gam = (255*(r/255)**0.4).astype(np.uint8)
out = cv2.LUT(img, gam)
# one memory lookup # per pixel
```

Parameters used —  $\gamma = 0.4$ ;  $c = 45.99$  (derived). Full program: L06\_intensity\_transforms.py

What it means — Precompute T once for 256 values; apply by lookup —  $O(1)$  per pixel.

Something to think about — 16-bit input? The LUT needs 65 536 entries — or compute directly in float.

## Choosing the transform

| Transform                    | Control                 | Use when                                       | Fails when                                             |
|------------------------------|-------------------------|------------------------------------------------|--------------------------------------------------------|
| Negative $s = 255 - r$       | none                    | Reading conventions (film, mammograms)         | Never 'fails' — but changes nothing structural         |
| Log $s = c \cdot \ln(1+r)$   | base/c                  | Huge dynamic range (spectra, HDR-ish)          | Bright-end detail matters — flat tail crushes it       |
| Gamma $s = c \cdot r^\gamma$ | $\gamma$                | Global brightness correction; display matching | Noise in dark regions — steep slope amplifies it       |
| Stretch (piecewise)          | $r_1, r_2$ break-points | Known usable band to expand                    | Outlier pixels define min/max — one hot pixel kills it |

*Key takeaway: every transform buys slope somewhere by selling it elsewhere — know what you are selling.*

## Core concepts

### Failure cases: the fine print

Noise amplification (gamma  $< 1$ , log) — The steep dark-end slope multiplies sensor noise along with signal — a dark pixel of noise  $\sigma = 3$  emerges with  $\sigma \approx 10$  after  $\gamma = 0.4$ . Under-exposed inputs get brighter AND grainier; this is the quantitative reason Unit IV low-light methods must jointly brighten and denoise (L33).

Outlier-driven stretching — min-max stretch computes its break-points from the data; a single specular highlight or dead pixel sets  $r_2 = 255$  and the stretch does nothing. Robust repair: use the 2nd and 98th percentiles.

Clipping — Any transform followed by uint8 casting silently saturates values outside  $[0, 255]$  — information destroyed, histograms pile up at 0 and 255. Always inspect the output histogram (L7 makes this systematic).

## Where this is used — your project

Low-light enhancement lane (primary) — gamma and log on the V channel are your REQUIRED classical baselines — L33's AI methods are graded against them, on the same images, with the same metrics; record  $\gamma$  and  $c$  in the dataset card so the comparison is reproducible.

Every other lane — percentile stretching is the standard display normalisation for results figures — use it (and say so) when showing outputs in reports.

Try this yourself — Apply  $\gamma \in \{0.3, 0.5, 0.7\}$  to your darkest project image; note where noise becomes visible before detail does.

## Summary

- A point operation is one curve applied everywhere; slope = local contrast gain. Buy slope where the task needs detail; the bill is paid at other intensities (and in noise). Gamma/log on V are the classical low-light baselines your AI methods must beat.

## Key formulae

$$s = (L-1) - r \quad s = c \cdot \ln(1 + r), \quad c = 255 / \ln 256 \approx 45.99 \quad s = 255 \cdot (r/255)^\gamma \quad (\gamma < 1 \text{ brightens}) \quad \text{Robust stretch: 2nd-98th percentile}$$

## What comes next

Before Friday — Nothing due — but run the ten-minute gamma sweep on your project image while today is fresh.

PCM-3 (releases Fri 14 Aug, 5 PM; due Mon 17 Aug, 8 PM) — Kernel prediction card + histogram interpretation (~30 min) — feeds L7, L8 and Tuesday's studio.

Logistics — Next lecture L7 (Tue 18 Aug, 12 PM): histogram analysis and equalisation. Tutorial T3 (Tue 18 Aug, 2-3 PM): spatial filtering and convolution studio.

## Check your understanding

Work these before the tutorial. They are the same questions used in class; the reasoning is discussed there, so attempt them first.

### 1. Predict before display

Three inputs, three transforms — match them before seeing any output: Inputs: (1) under-exposed night photo; (2) Fourier magnitude spectrum with one huge DC value; (3) chest X-ray where dense tissue prints bright. Transforms: negative · log · gamma ( $\gamma < 1$ ). For each pairing, predict what the histogram does.

## Practice questions

Attempt these unaided, then compare with the discussion in your tutorial session.

Q1 (4 marks, remember/understand): Define a point operation. Why does monotonicity matter, and give one transform from this lecture that is monotone decreasing.

Q2 (6 marks, apply): Derive the constant  $c$  for a full-range log transform on 8-bit data, then compute the outputs of  $r = 100$  under that transform and  $r = 200$  under  $\gamma = 2.2$ .

Q3 (10 marks, analyse): A colleague brightens under-exposed microscopy images with  $\gamma = 0.3$  and reports that 'the images look brighter but the analysis pipeline now finds more false cells'. Explain

the mechanism and propose two remedies with their trade-offs.

## **Further reading**

GW4 Ch.3 Sec.3.2; Szeliski Ch.3 Sec.3.1

Course notes: Comprehensive Notes L7; Vol. I ch. 7.

Runnable code for every figure and number in this lecture: 06\_CODE\_AND\_DEMOS (see the file names quoted beside each figure).

*AI531C Digital Image Analysis · student handout · every figure and numerical result in this document is the output of course code you can run yourself.*